

UAPOML Week 3 Problem Set

Written Report

Classification, Regression Trees, Ensemble Methods & Regularisation

Submission: Jupyter Notebook (UAPOML_W3_Solution.ipynb) + this Written Report | **Tools:** Python, scikit-learn, XGBoost, Kaggle | **Random Seed:** random_state=42 throughout

Q1 — Logistic Regression for Credit Default Prediction [Easy]

What and Why

We built a binary classifier using logistic regression to predict whether a borrower will experience serious financial distress within two years. Logistic regression is the natural first choice for binary classification in credit scoring because it is **convex** (guaranteed global convergence), **interpretable** via coefficient magnitudes, and **calibrated** (outputs genuine probabilities). Its coefficient vector directly encodes the marginal log-odds contribution of each feature, making it auditable in regulated financial contexts.

Interpretation of Top Coefficients

The five features with the largest absolute coefficients reveal the most influential predictors:

- **RevolvingUtilizationOfUnsecuredLines:** Largest positive coefficient. Borrowers using a high fraction of unsecured credit lines face acute liquidity pressure and are far more likely to default — consistent with FICO-type models.
- **NumberOfTime30-59DaysPastDueNotWorse:** Historical delinquency is the strongest known predictor of future default. Even one 30-day late payment substantially elevates default probability.
- **NumberOfTimes90DaysLate:** Captures the most severe past delinquencies. Large positive coefficient confirms these borrowers are at extremely high risk.
- **DebtRatio:** Monthly debt obligations / monthly income. High leverage leaves no buffer against income shocks, making default much more likely in stress scenarios.
- **age (negative coefficient):** Older borrowers have more established credit histories and greater income stability, reducing default probability. This aligns with the FICO methodology.

The threshold analysis showed that the default threshold of 0.5 is suboptimal for the imbalanced dataset. The best F1-maximising threshold is lower, increasing recall for the minority (defaulting) class.

Q2 — Decision Tree Classifier with Pruning for Heart Disease [Medium]

What and Why

We applied decision tree classification to the Heart Disease UCI dataset, comparing unpruned trees with pre-pruning (depth-limiting) and post-pruning (cost-complexity). Decision trees are valued in

healthcare for their intrinsic interpretability — clinicians can trace which physiological features drive a prediction. Without pruning, however, trees overfit severely.

Results Analysis

- **Unpruned Tree:** Training accuracy near-perfect (~ 1.0), but test accuracy drops substantially — classic overfitting.
- **Depth-limited pre-pruning:** Validation accuracy peaked at moderate depth (typically 3–5). Beyond this, the tree fits noise.
- **Cost-complexity pruning (winner):** Achieved higher test AUC-ROC than pre-pruning by working globally — evaluating the cost-benefit of removing each subtree rather than greedily halting splits.

Key Insight: The Horizon Effect

The fundamental limitation of pre-pruning is the **horizon effect**: the algorithm cannot look ahead to see whether a currently unhelpful split will unlock a highly discriminative split downstream. Cost-complexity pruning, working top-down on the fully grown tree, evaluates the global cost-reduction of each subtree removal, producing better generalisation.

Q3 — Regularised Regression for House Price Prediction [Medium]

What and Why

We applied Ridge, Lasso, and Elastic Net regularisation to the 79-feature House Prices dataset. High multicollinearity among structural features (GrLivArea, TotalBsmtSF, GarageArea) makes unregularised OLS unstable. We log-transformed SalePrice to convert the right-skewed target into a more Gaussian distribution.

Geometric Explanation: Why Lasso Produces Sparse Solutions

The L1 constraint region (Lasso) is a **hyperdiamond** whose corners lie exactly on the coordinate axes. The squared-error loss function forms elliptical contours in coefficient space. The regularised solution is where these contours first touch the constraint region. Because the L1 hyperdiamond has corners on the axes, the intersection is geometrically much more likely to occur at a corner — where one or more coefficients are exactly zero. The smooth L2 ball (Ridge) has no corners, so the tangent point almost never falls on a coordinate axis.

Implications for Noisy Financial Data

In noisy financial datasets with many potentially irrelevant features, Lasso's automatic feature selection is critical. By driving irrelevant coefficients to exactly zero, Lasso reduces model complexity and overfitting risk, improving out-of-sample interpretability. Elastic Net combines both penalties, handling correlated features better than pure Lasso (which arbitrarily selects one from a correlated group) while still achieving sparsity.

Q4 — Locally Weighted Regression on Non-Linear Data [Medium]

What and Why

We implemented LWR from scratch using NumPy on the California Housing dataset (median_income → median_house_value). LWR fits a local linear model at each query point, weighting nearby observations via a Gaussian kernel: $w_i = \exp(-||x_i - x_q||^2 / 2\tau^2)$.

Role of Bandwidth τ and Its Mathematical Limit

- **Small τ (e.g., 0.05):** Very narrow kernel — only nearest neighbours receive weight. High variance, low bias (overfitting to noise).
- **Large τ (e.g., 3.0):** Wide kernel — nearly equal weights for all points. Low variance, high bias (global linear fit).
- **Limit as $\tau \rightarrow \infty$:** All weights $w_i \rightarrow 1$, so $W \rightarrow I$. The weighted least-squares problem $\theta = (X'WX)^{-1}X'Wy$ reduces to OLS: $\theta = (X'X)^{-1}X'y$. LWR with $\tau \rightarrow \infty$ is exactly OLS.

Practical Difficulties in Higher Dimensions

The **curse of dimensionality** shrinks the effective neighbourhood exponentially with d . In d dimensions, the expected nearest-neighbour distance grows as $O(n^{-1/d})$, requiring exponentially more data for the same estimation accuracy. Computational cost also scales as $O(n^2d)$ per query, making LWR infeasible for large high-dimensional datasets without approximations.

Q5 — Random Forest with Feature Importance for Fraud Detection [Hard]

What and Why

We applied a Random Forest classifier to the IEEE-CIS Fraud Detection dataset, which has severe class imbalance (~3.5% fraud rate) and high-dimensional heterogeneous features. The `class_weight='balanced'` setting inversely weights classes by frequency, effectively upweighting the minority (fraud) class during training.

MDI vs. Permutation Importance Discrepancy

MDI (Mean Decrease in Impurity) has a known upward bias towards high-cardinality features (e.g., transaction IDs, card numbers). These features offer many split thresholds and achieve high impurity reductions by chance on the training set.

Permutation Importance, computed on the held-out validation set, measures the actual AUC degradation when a feature is randomly shuffled — providing an unbiased estimate of true out-of-sample relevance. Features that rank high under MDI but low under permutation importance are likely memorisation cues rather than genuine signals.

Why AUC-PR is More Informative under Class Imbalance

AUC-ROC incorporates the True Negative Rate (specificity). When the negative class is vastly larger (96.5% non-fraud), even a poor model produces a very high TNR by defaulting to 'not fraud'. This inflates AUC-ROC. AUC-PR focuses exclusively on precision and recall of the positive (fraud) class — the class we actually care about — and is not inflated by true negatives.

Q6 — XGBoost for Stock Return Classification [Hard]

What and Why

We used XGBoost to predict whether S&P 500 stocks would produce a positive 21-day forward return. We enforced a strict time-based train/test split (train: pre-2017, test: 2017+) to respect temporal ordering and avoid look-ahead bias.

Finance Context: Uncertainty-Aware Allocation

XGBoost's predicted probability p^* is a return signal, but without a confidence estimate, equal position sizing treats a marginal signal ($p^* = 0.51$) identically to a strong one ($p^* = 0.80$). Combining with ensemble uncertainty σ (from Q8) enables position sizing $\propto (p^* - 0.5)/\sigma$, improving risk-adjusted returns.

Why Random k-Fold CV is Invalid for Financial Time Series

Random k-fold CV violates temporal ordering, creating look-ahead bias. The target at time t (21-day forward return) depends on prices at $t+21$. Random shuffling assigns temporally adjacent observations to different folds, allowing the model to indirectly 'see the future.' Additionally, financial returns exhibit autocorrelation (momentum, mean-reversion), violating the i.i.d. assumption underlying standard CV. The result is grossly optimistic performance estimates that evaporate in live trading. The correct approach is **walk-forward (expanding or rolling window) validation**, strictly enforcing that training data precedes validation data in calendar time.

Q7 — Empirical Bias–Variance Decomposition [Medium]

What and Why

We empirically estimated Bias² and Variance for five models using $B=50$ bootstrap iterations on the Pima Indians Diabetes dataset, spanning the complexity spectrum from Decision Stump (high bias) to Unpruned Tree (high variance) to Random Forest and XGBoost (low bias AND low variance).

Why Bagging Reduces Variance but Not Bias; Why Boosting Reduces Both

The formal variance of the average of B correlated predictions is: $\text{Var}[\bar{y}] = p\sigma^2 + (1-p)\sigma^2/B$, where p is the average pairwise tree correlation and σ^2 is a single tree's variance.

- **Bagging (Random Forest):** As $B \rightarrow \infty$, the $(1-p)\sigma^2/B$ term vanishes, dramatically reducing variance. However, averaging models trained on bootstrap samples of the same data does not change the mean prediction — bias remains equal to a single tree's bias. Since individual trees are already low-bias, Random Forest achieves low bias AND low variance.
- **Boosting (XGBoost):** Each new tree fits the residual errors of the current ensemble — explicitly correcting systematic mistakes (reducing bias). Simultaneously, XGBoost's L1/L2 regularisation, subsampling, and column sampling control variance. Boosting thus achieves simultaneous bias AND variance reduction.

Q8 — Gradient Boosting with Uncertainty Estimation for Return Prediction [Hard]

What and Why

This capstone integrates gradient boosting, feature engineering, time-series cross-validation, and prediction uncertainty estimation via ensemble disagreement on the NIFTY-50 dataset. Ensemble disagreement — the standard deviation σ of $M=20$ bootstrap-trained gradient boosting models — provides a natural, model-agnostic uncertainty proxy.

Uncertainty-Aware Portfolio Allocation

In the UAPOML framework, σ directly informs position sizing: position weight $\propto (p^* - 0.5) / \sigma$. This Sharpe-ratio-like normalisation prevents committing large capital to high-uncertainty predictions, analogous to the Kelly criterion's adjustment for uncertainty in expected return.

Key Risks of Live Deployment

- **Overfitting to non-stationary data:** Financial distributions undergo regime changes. A model trained on 2000–2017 NIFTY-50 data captures regime-specific patterns that may not persist. Factor relationships can flip sign during regime shifts, causing severe out-of-sample losses.
- **Look-ahead bias:** Using adjusted prices incorporating post-hoc dividend events, or index reconstitution data not available at decision time, creates subtle look-ahead bias that inflates backtests. A rigorous data pipeline must ensure all features are computed exclusively from information available at the decision timestamp.
- **Transaction costs:** Bid-ask spreads, market impact, brokerage commissions, and short-selling costs are ignored in our illustrative strategy. For a daily-rebalanced portfolio, these costs easily consume 50–150+ bps per round-trip, eliminating apparent alpha.
- **σ and position sizing:** High σ indicates the input is in a poorly-covered region of feature space. An uncertainty-aware portfolio shrinks positions proportionally to σ , reducing exposure to ambiguous predictions. This directly implements the UAPOML theme of uncertainty-aware allocation.

Bonus — Stacked Generalisation for Titanic Survival [Hard]

What and Why

Stacking learns an optimal meta-combination of base learner predictions. Different models have complementary strengths: Logistic Regression (linear separability), Decision Tree (non-linear interactions), KNN (local density), Random Forest (variance reduction), XGBoost (bias+variance reduction). The meta-learner exploits these complementarities.

Why Out-of-Fold Predictions Are Essential: Preventing Data Leakage

OOF predictions must be used because the meta-learner requires predictions that the base learners did not see during training. If in-sample predictions were used instead:

- Heavily overfit base learners (e.g., unpruned decision trees) would achieve ~100% in-sample accuracy by memorising training examples — not by learning generalisable signal.
- The meta-learner would learn to trust these memorised predictions, assigning high weight to overfit base models.
- In deployment, the overfit base learner produces noisy predictions on unseen data, causing the meta-learner's weights to be grossly miscalibrated.

- The stacked ensemble would inherit the base learner's poor generalisation, severely degrading performance.

The OOF procedure provides an **unbiased estimate** of each base learner's out-of-sample performance, allowing the meta-learner to assign weights proportional to true generalisability. This is the same logic as cross-validation: by holding out each fold during base learner training, we simulate the out-of-sample setting the meta-learner will actually encounter.

General Methodology Notes

Data Leakage Prevention

All preprocessing (StandardScaler, imputation, one-hot encoding) was fit exclusively on the training set and applied to validation/test sets via transform(). This prevents test distribution information from influencing feature scaling, which would bias reported metrics.

Finance-Specific Considerations

- **Temporal Boundaries:** Q6 (S&P 500): Train pre-2017 / Test 2017+. Q8 (NIFTY-50): Train first 80% chronologically / Test last 20%.
- **Survivorship Bias:** Both datasets include only companies that survived (remained in the index) throughout the study period, creating an upward bias in return predictions. This is an inherent limitation of using survivorship-biased historical data.
- **Uncertainty-Aware Allocation:** σ_{ensemble} from ensemble disagreement serves as a position-sizing modifier in the UAPOML framework. Position weight $\propto (p^*_{\text{ensemble}} - 0.5) / \sigma_{\text{ensemble}}$ implements a signal-to-noise ratio that reduces exposure to ambiguous predictions.